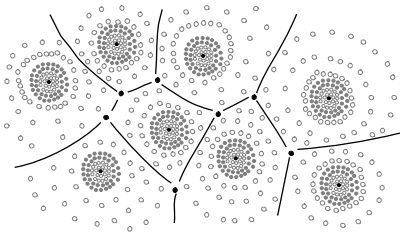
Chapter 2 Object-Oriented Programming



Contents

2.1 Goals, Principles, and Patterns 57
2.1.1 Object-Oriented Design Goals 57
2.1.2 Object-Oriented Design Principles 58
2.1.3 Design Patterns 61
2.2 Software Development 62
2.2.1 Design 62
2.2.2 Pseudo-Code 64
2.2.3 Coding Style and Documentation 64
2.2.4 Testing and Debugging 67
2.3 Class Definitions 69
2.3.1 Example: CreditCard Class 69
2.3.2 Operator Overloading and Python's Special Methods . . . 74
2.3.3 Example: Multidimensional Vector Class 77
2.3.4 Iterators 79
2.3.5 Example: Range Class 80
2.4 Inheritance 82
2.4.1 Extending the CreditCard Class 83
2.4.2 Hierarchy of Numeric Progressions 87
2.4.3 Abstract Base Classes 93
2.5 Namespaces and Object-Orientation 96
2.5.1 Instance and Class Namespaces 96
2.5.2 Name Resolution and Dynamic Dispatch 100
2.6 Shallow and Deep Copying 101
2.7 Exercises 103

2.1 Goals, Principles, and Patterns

As the name implies, the main "actors" in the object-oriented paradigm are called ***objects***. Each object is an ***instance*** of a ***class***. Each class presents to the outside world a concise and consistent view of the objects that are instances of this class, without going into too much unnecessary detail or giving others access to the inner workings of the objects. The class definition typically specifies ***instance variables***, also known as ***data members***, that the object contains, as well as the ***methods***, also known as ***member functions***, that the object can execute. This view of computing is intended to fulfill several goals and incorporate several design principles, which we discuss in this chapter.

2.1.1 Object-Oriented Design Goals

Software implementations should achieve ***robustness***, ***adaptability***, and ***reusability***. (See Figure 2.1.)

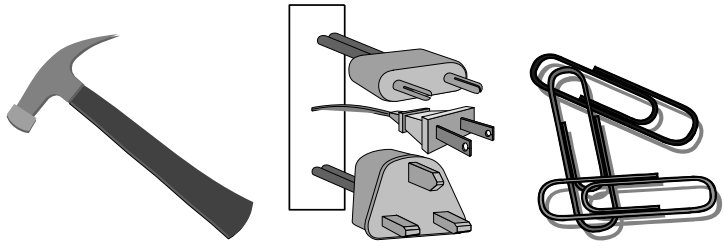


Figure 2.1: Goals of object-oriented design.

Robustness

Every good programmer wants to develop software that is correct, which means that a program produces the right output for all the anticipated inputs in the program's application. In addition, we want software to be ***robust***, that is, capable of handling unexpected inputs that are not explicitly defined for its application. For example, if a program is expecting a positive integer (perhaps representing the price of an item) and instead is given a negative integer, then the program should be able to recover gracefully from this error. More importantly, in ***life-critical applications***, where a software error can lead to injury or loss of life, software that is not robust could be deadly. This point was driven home in the late 1980s in accidents involving Therac-25, a radiation-therapy machine, which severely overdosed six patients between 1985 and 1987, some of whom died from complications resulting from their radiation overdose. All six accidents were traced to software errors.

Adaptability

Modern software applications, such as Web browsers and Internet search engines, typically involve large programs that are used for many years. Software, therefore, needs to be able to evolve over time in response to changing conditions in its environment. Thus, another important goal of quality software is that it achieves ***adaptability*** (also called ***evolvability***). Related to this concept is ***portability***, which is the ability of software to run with minimal change on different hardware and operating system platforms. An advantage of writing software in Python is the portability provided by the language itself.

Reusability

Going hand in hand with adaptability is the desire that software be reusable, that is, the same code should be usable as a component of different systems in various applications. Developing quality software can be an expensive enterprise, and its cost can be offset somewhat if the software is designed in a way that makes it easily reusable in future applications. Such reuse should be done with care, however, for one of the major sources of software errors in the Therac-25 came from inappropriate reuse of Therac-20 software (which was not object-oriented and not designed for the hardware platform used with the Therac-25).

2.1.2 Object-Oriented Design Principles

Chief among the principles of the object-oriented approach, which are intended to facilitate the goals outlined above, are the following (see Figure 2.2):

- Modularity
- Abstraction
- Encapsulation

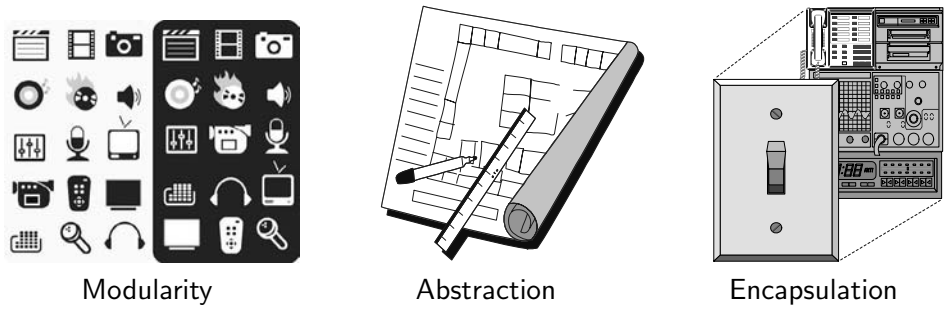


Figure 2.2: Principles of object-oriented design.

Modularity

Modern software systems typically consist of several different components that must interact correctly in order for the entire system to work properly. Keeping these interactions straight requires that these different components be well organized. Modularity refers to an organizing principle in which different components of a software system are divided into separate functional units.

As a real-world analogy, a house or apartment can be viewed as consisting of several interacting units: electrical, heating and cooling, plumbing, and structural. Rather than viewing these systems as one giant jumble of wires, vents, pipes, and boards, the organized architect designing a house or apartment will view them as separate modules that interact in well-defined ways. In so doing, he or she is using modularity to bring a clarity of thought that provides a natural way of organizing functions into distinct manageable units.

In like manner, using modularity in a software system can also provide a powerful organizing framework that brings clarity to an implementation. In Python, we have already seen that a ***module*** is a collection of closely related functions and classes that are defined together in a single file of source code. Python's standard libraries include, for example, the `math` module, which provides definitions for key mathematical constants and functions, and the `os` module, which provides support for interacting with the operating system.

The use of modularity helps support the goals listed in Section 2.1.1. Robustness is greatly increased because it is easier to test and debug separate components before they are integrated into a larger software system. Furthermore, bugs that persist in a complete system might be traced to a particular component, which can be fixed in relative isolation. The structure imposed by modularity also helps enable software reusability. If software modules are written in a general way, the modules can be reused when related need arises in other contexts. This is particularly relevant in a study of data structures, which can typically be designed with sufficient abstraction and generality to be reused in many applications.

Abstraction

The notion of ***abstraction*** is to distill a complicated system down to its most fundamental parts. Typically, describing the parts of a system involves naming them and explaining their functionality. Applying the abstraction paradigm to the design of data structures gives rise to ***abstract data types*** (ADTs). An ADT is a mathematical model of a data structure that specifies the type of data stored, the operations supported on them, and the types of parameters of the operations. An ADT specifies ***what*** each operation does, but not ***how*** it does it. We will typically refer to the collective set of behaviors supported by an ADT as its ***public interface***.

As a programming language, Python provides a great deal of latitude in regard to the specification of an interface. Python has a tradition of treating abstractions implicitly using a mechanism known as ***duck typing***. As an interpreted and dynamically typed language, there is no "compile time" checking of data types in Python, and no formal requirement for declarations of abstract base classes. Instead programmers assume that an object supports a set of known behaviors, with the interpreter raising a run-time error if those assumptions fail. The description of this as "duck typing" comes from an adage attributed to poet James Whitcomb Riley, stating that "when I see a bird that walks like a duck and swims like a duck and quacks like a duck, I call that bird a duck."

More formally, Python supports abstract data types using a mechanism known as an ***abstract base class*** (ABC). An abstract base class cannot be instantiated (i.e., you cannot directly create an instance of that class), but it defines one or more common methods that all implementations of the abstraction must have. An ABC is realized by one or more ***concrete classes*** that inherit from the abstract base class while providing implementations for those method declared by the ABC. Python's `abc` module provides formal support for ABCs, although we omit such declarations for simplicity. We will make use of several existing abstract base classes coming from Python's `collections` module, which includes definitions for several common data structure ADTs, and concrete implementations of some of those abstractions.

Encapsulation

Another important principle of object-oriented design is ***encapsulation***. Different components of a software system should not reveal the internal details of their respective implementations. One of the main advantages of encapsulation is that it gives one programmer freedom to implement the details of a component, without concern that other programmers will be writing code that intricately depends on those internal decisions. The only constraint on the programmer of a component is to maintain the public interface for the component, as other programmers will be writing code that depends on that interface. Encapsulation yields robustness and adaptability, for it allows the implementation details of parts of a program to change without adversely affecting other parts, thereby making it easier to fix bugs or add new functionality with relatively local changes to a component.

Throughout this book, we will adhere to the principle of encapsulation, making clear which aspects of a data structure are assumed to be public and which are assumed to be internal details. With that said, Python provides only loose support for encapsulation. By convention, names of members of a class (both data members and member functions) that start with a single underscore character (e.g., `_secret`) are assumed to be nonpublic and should not be relied upon. Those conventions are reinforced by the intentional omission of those members from automatically generated documentation.

2.1.3 Design Patterns

Object-oriented design facilitates reusable, robust, and adaptable software. Designing good code takes more than simply understanding object-oriented methodologies, however. It requires the effective use of object-oriented design techniques.

Computing researchers and practitioners have developed a variety of organizational concepts and methodologies for designing quality object-oriented software that is concise, correct, and reusable. Of special relevance to this book is the concept of a ***design pattern***, which describes a solution to a "typical" software design problem. A pattern provides a general template for a solution that can be applied in many different situations. It describes the main elements of a solution in an abstract way that can be specialized for a specific problem at hand. It consists of a name, which identifies the pattern; a context, which describes the scenarios for which this pattern can be applied; a template, which describes how the pattern is applied; and a result, which describes and analyzes what the pattern produces.

We present several design patterns in this book, and we show how they can be consistently applied to implementations of data structures and algorithms. These design patterns fall into two groups—patterns for solving algorithm design problems and patterns for solving software engineering problems. The algorithm design patterns we discuss include the following:

- Recursion (Chapter 4)
- Amortization (Sections 5.3 and 11.4)
- Divide-and-conquer (Section 12.2.1)
- Prune-and-search, also known as decrease-and-conquer (Section 12.7.1)
- Brute force (Section 13.2.1)
- Dynamic programming (Section 13.3).
- The greedy method (Sections 13.4.2, 14.6.2, and 14.7)

Likewise, the software engineering design patterns we discuss include:

- Iterator (Sections 1.8 and 2.3.4)
- Adapter (Section 6.1.2)
- Position (Sections 7.4 and 8.1.2)
- Composition (Sections 7.6.1, 9.2.1, and 10.1.4)
- Template method (Sections 2.4.3, 8.4.6, 10.1.3, 10.5.2, and 11.2.1)
- Locator (Section 9.5.1)
- Factory method (Section 11.2.1)

Rather than explain each of these concepts here, however, we introduce them throughout the text as noted above. For each pattern, be it for algorithm engineering or software engineering, we explain its general use and we illustrate it with at least one concrete example.

2.2 Software Development

Traditional software development involves several phases. Three major steps are:

1. Design
2. Implementation
3. Testing and Debugging

In this section, we briefly discuss the role of these phases, and we introduce several good practices for programming in Python, including coding style, naming conventions, formal documentation, and unit testing.

2.2.1 Design

For object-oriented programming, the design step is perhaps the most important phase in the process of developing software. For it is in the design step that we decide how to divide the workings of our program into classes, we decide how these classes will interact, what data each will store, and what actions each will perform. Indeed, one of the main challenges that beginning programmers face is deciding what classes to define to do the work of their program. While general prescriptions are hard to come by, there are some rules of thumb that we can apply when determining how to design our classes:

- ***Responsibilities***: Divide the work into different ***actors***, each with a different responsibility. Try to describe responsibilities using action verbs. These actors will form the classes for the program.
- ***Independence***: Define the work for each class to be as independent from other classes as possible. Subdivide responsibilities between classes so that each class has autonomy over some aspect of the program. Give data (as instance variables) to the class that has jurisdiction over the actions that require access to this data.
- ***Behaviors***: Define the behaviors for each class carefully and precisely, so that the consequences of each action performed by a class will be well understood by other classes that interact with it. These behaviors will define the methods that this class performs, and the set of behaviors for a class are the ***interface*** to the class, as these form the means for other pieces of code to interact with objects from the class.

Defining the classes, together with their instance variables and methods, are key to the design of an object-oriented program. A good programmer will naturally develop greater skill in performing these tasks over time, as experience teaches him or her to notice patterns in the requirements of a program that match patterns that he or she has seen before.

A common tool for developing an initial high-level design for a project is the use of ***CRC cards***. Class-Responsibility-Collaborator (CRC) cards are simple index cards that subdivide the work required of a program. The main idea behind this tool is to have each card represent a component, which will ultimately become a class in the program. We write the name of each component on the top of an index card. On the left-hand side of the card, we begin writing the responsibilities for this component. On the right-hand side, we list the collaborators for this component, that is, the other components that this component will have to interact with to perform its duties.

The design process iterates through an action/actor cycle, where we first identify an action (that is, a responsibility), and we then determine an actor (that is, a component) that is best suited to perform that action. The design is complete when we have assigned all actions to actors. In using index cards for this process (rather than larger pieces of paper), we are relying on the fact that each component should have a small set of responsibilities and collaborators. Enforcing this rule helps keep the individual classes manageable.

As the design takes form, a standard approach to explain and document the design is the use of UML (Unified Modeling Language) diagrams to express the organization of a program. UML diagrams are a standard visual notation to express object-oriented software designs. Several computer-aided tools are available to build UML diagrams. One type of UML figure is known as a ***class diagram***. An example of such a diagram is given in Figure 2.3, for a class that represents a consumer credit card. The diagram has three portions, with the first designating the name of the class, the second designating the recommended instance variables, and the third designating the recommended methods of the class. In Section 2.2.3, we discuss our naming conventions, and in Section 2.3.1, we provide a complete implementation of a Python CreditCard class based on this design.

Class:	CreditCard	
Fields:	_customer _bank _account	_balance _limit
Behaviors:	get_customer( ) get_bank( ) get_account( ) make_payment(amount)	get_balance( ) get_limit( ) charge(price)

Figure 2.3: Class diagram for a proposed CreditCard class.

2.2.2 Pseudo-Code

As an intermediate step before the implementation of a design, programmers are often asked to describe algorithms in a way that is intended for human eyes only. Such descriptions are called ***pseudo-code***. Pseudo-code is not a computer program, but is more structured than usual prose. It is a mixture of natural language and high-level programming constructs that describe the main ideas behind a generic implementation of a data structure or algorithm. Because pseudo-code is designed for a human reader, not a computer, we can communicate high-level ideas, without being burdened with low-level implementation details. At the same time, we should not gloss over important steps. Like many forms of human communication, finding the right balance is an important skill that is refined through practice.

In this book, we rely on a pseudo-code style that we hope will be evident to Python programmers, yet with a mix of mathematical notations and English prose. For example, we might use the phrase "indicate an error" rather than a formal raise statement. Following conventions of Python, we rely on indentation to indicate the extent of control structures and on an indexing notation in which entries of a sequence A with length n are indexed from $A[0]$ to $A[n-1]$. However, we choose to enclose comments within curly braces { like these } in our pseudo-code, rather than using Python's # character.

2.2.3 Coding Style and Documentation

Programs should be made easy to read and understand. Good programmers should therefore be mindful of their coding style, and develop a style that communicates the important aspects of a program's design for both humans and computers. Conventions for coding style tend to vary between different programming communities. The official *Style Guide for Python Code* is available online at

`http://www.python.org/dev/peps/pep-0008/`

The main principles that we adopt are as follows:

- Python code blocks are typically indented by 4 spaces. However, to avoid having our code fragments overrun the book's margins, we use 2 spaces for each level of indentation. It is strongly recommended that tabs be avoided, as tabs are displayed with differing widths across systems, and tabs and spaces are not viewed as identical by the Python interpreter. Many Python-aware editors will automatically replace tabs with an appropriate number of spaces.

- Use meaningful names for identifiers. Try to choose names that can be read aloud, and choose names that reflect the action, responsibility, or data each identifier is naming.

  - Classes (other than Python's built-in classes) should have a name that serves as a singular noun, and should be capitalized (e.g., `Date` rather than `date` or `Dates`). When multiple words are concatenated to form a class name, they should follow the so-called "CamelCase" convention in which the first letter of each word is capitalized (e.g., `CreditCard`).

  - Functions, including member functions of a class, should be lowercase. If multiple words are combined, they should be separated by underscores (e.g., `make_payment`). The name of a function should typically be a verb that describes its affect. However, if the only purpose of the function is to return a value, the function name may be a noun that describes the value (e.g., `sqrt` rather than `calculate_sqrt`).

  - Names that identify an individual object (e.g., a parameter, instance variable, or local variable) should be a lowercase noun (e.g., `price`). Occasionally, we stray from this rule when using a single uppercase letter to designate the name of a data structures (such as tree `T`).

  - Identifiers that represent a value considered to be a constant are traditionally identified using all capital letters and with underscores to separate words (e.g., `MAX_SIZE`).

  Recall from our discussion of ***encapsulation*** that identifiers in any context that begin with a single leading underscore (e.g., `_secret`) are intended to suggest that they are only for "internal" use to a class or module, and not part of a public interface.

- Use comments that add meaning to a program and explain ambiguous or confusing constructs. In-line comments are good for quick explanations; they are indicated in Python following the # character, as in

  ```
  if n % 2 == 1:          # n is odd
  ```

  Multiline block comments are good for explaining more complex code sections. In Python, these are technically multiline string literals, typically delimited with triple quotes (`"""`), which have no effect when executed. In the next section, we discuss the use of block comments for documentation.

Documentation

Python provides integrated support for embedding formal documentation directly in source code using a mechanism known as a ***docstring***. Formally, any string literal that appears as the *first* statement within the body of a module, class, or function (including a member function of a class) will be considered to be a docstring. By convention, those string literals should be delimited within triple quotes ("""). As an example, our version of the scale function from page 25 could be documented as follows:

```
def scale(data, factor):
  """Multiply all entries of numeric data list by the given factor."""
  for j in range(len(data)):
    data[j] *= factor
```

It is common to use the triple-quoted string delimiter for a docstring, even when the string fits on a single line, as in the above example. More detailed docstrings should begin with a single line that summarizes the purpose, followed by a blank line, and then further details. For example, we might more clearly document the scale function as follows:

```
def scale(data, factor):
  """Multiply all entries of numeric data list by the given factor.

  data    an instance of any mutable sequence type (such as a list)
          containing numeric elements

  factor  a number that serves as the multiplicative factor for scaling
  """
  for j in range(len(data)):
    data[j] *= factor
```

A docstring is stored as a field of the module, function, or class in which it is declared. It serves as documentation and can be retrieved in a variety of ways. For example, the command help(x), within the Python interpreter, produces the documentation associated with the identified object x. An external tool named pydoc is distributed with Python and can be used to generate formal documentation as text or as a Web page. Guidelines for *authoring* useful docstrings are available at:

`http://www.python.org/dev/peps/pep-0257/`

In this book, we will try to present docstrings when space allows. Omitted docstrings can be found in the online version of our source code.

2.2.4 Testing and Debugging

Testing is the process of experimentally checking the correctness of a program, while debugging is the process of tracking the execution of a program and discovering the errors in it. Testing and debugging are often the most time-consuming activity in the development of a program.

Testing

A careful testing plan is an essential part of writing a program. While verifying the correctness of a program over all possible inputs is usually infeasible, we should aim at executing the program on a representative subset of inputs. At the very minimum, we should make sure that every method of a class is tested at least once (method coverage). Even better, each code statement in the program should be executed at least once (statement coverage).

Programs often tend to fail on ***special cases*** of the input. Such cases need to be carefully identified and tested. For example, when testing a method that sorts (that is, puts in order) a sequence of integers, we should consider the following inputs:

- The sequence has zero length (no elements).
- The sequence has one element.
- All the elements of the sequence are the same.
- The sequence is already sorted.
- The sequence is reverse sorted.

In addition to special inputs to the program, we should also consider special conditions for the structures used by the program. For example, if we use a Python list to store data, we should make sure that boundary cases, such as inserting or removing at the beginning or end of the list, are properly handled.

While it is essential to use handcrafted test suites, it is also advantageous to run the program on a large collection of randomly generated inputs. The `random` module in Python provides several means for generating random numbers, or for randomizing the order of collections.

The dependencies among the classes and functions of a program induce a hierarchy. Namely, a component A is above a component B in the hierarchy if A depends upon B, such as when function A calls function B, or function A relies on a parameter that is an instance of class B. There are two main testing strategies, ***top-down*** and ***bottom-up***, which differ in the order in which components are tested.

Top-down testing proceeds from the top to the bottom of the program hierarchy. It is typically used in conjunction with ***stubbing***, a boot-strapping technique that replaces a lower-level component with a ***stub***, a replacement for the component that simulates the functionality of the original. For example, if function A calls function B to get the first line of a file, when testing A we can replace B with a stub that returns a fixed string.

Bottom-up testing proceeds from lower-level components to higher-level components. For example, bottom-level functions, which do not invoke other functions, are tested first, followed by functions that call only bottom-level functions, and so on. Similarly a class that does not depend upon any other classes can be tested before another class that depends on the former. This form of testing is usually described as ***unit testing***, as the functionality of a specific component is tested in isolation of the larger software project. If used properly, this strategy better isolates the cause of errors to the component being tested, as lower-level components upon which it relies should have already been thoroughly tested.

Python provides several forms of support for automated testing. When functions or classes are defined in a module, testing for that module can be embedded in the same file. The mechanism for doing so was described in Section 1.11. Code that is shielded in a conditional construct of the form

```
if __name__ == '__main__':
  # perform tests...
```

will be executed when Python is invoked directly on that module, but not when the module is imported for use in a larger software project. It is common to put tests in such a construct to test the functionality of the functions and classes specifically defined in that module.

More robust support for automation of unit testing is provided by Python's `unittest` module. This framework allows the grouping of individual test cases into larger test suites, and provides support for executing those suites, and reporting or analyzing the results of those tests. As software is maintained, the act of ***regression testing*** is used, whereby all previous tests are re-executed to ensure that changes to the software do not introduce new bugs in previously tested components.

Debugging

The simplest debugging technique consists of using ***print statements*** to track the values of variables during the execution of the program. A problem with this approach is that eventually the print statements need to be removed or commented out, so they are not executed when the software is finally released.

A better approach is to run the program within a ***debugger***, which is a specialized environment for controlling and monitoring the execution of a program. The basic functionality provided by a debugger is the insertion of ***breakpoints*** within the code. When the program is executed within the debugger, it stops at each breakpoint. While the program is stopped, the current value of variables can be inspected.

The standard Python distribution includes a module named `pdb`, which provides debugging support directly within the interpreter. Most IDEs for Python, such as IDLE, provide debugging environments with graphical user interfaces.

2.3 Class Definitions

A class serves as the primary means for abstraction in object-oriented programming. In Python, every piece of data is represented as an instance of some class. A class provides a set of behaviors in the form of ***member functions*** (also known as ***methods***), with implementations that are common to all instances of that class. A class also serves as a blueprint for its instances, effectively determining the way that state information for each instance is represented in the form of ***attributes*** (also known as ***fields***, ***instance variables***, or ***data members***).

2.3.1 Example: CreditCard Class

As a first example, we provide an implementation of a CreditCard class based on the design we introduced in Figure 2.3 of Section 2.2.1. The instances defined by the CreditCard class provide a simple model for traditional credit cards. They have identifying information about the customer, bank, account number, credit limit, and current balance. The class restricts charges that would cause a card's balance to go over its spending limit, but it does not charge interest or late payments (we revisit such themes in Section 2.4.1).

Our code begins in Code Fragment 2.1 and continues in Code Fragment 2.2. The construct begins with the keyword, **class**, followed by the name of the class, a colon, and then an indented block of code that serves as the body of the class. The body includes definitions for all methods of the class. These methods are defined as functions, using techniques introduced in Section 1.5, yet with a special parameter, named **self**, that serves to identify the particular instance upon which a member is invoked.

The self Identifier

In Python, the self identifier plays a key role. In the context of the CreditCard class, there can presumably be many different CreditCard instances, and each must maintain its own balance, its own credit limit, and so on. Therefore, each instance stores its own instance variables to reflect its current state.

Syntactically, self identifies the instance upon which a method is invoked. For example, assume that a user of our class has a variable, my_card, that identifies an instance of the CreditCard class. When the user calls my_card.get_balance( ), identifier self, within the definition of the get_balance method, refers to the card known as my_card by the caller. The expression, self._balance refers to an instance variable, named _balance, stored as part of that particular credit card's state.

```
1  class CreditCard:
2    """A consumer credit card."""
3
4    def __init__(self, customer, bank, acnt, limit):
5      """Create a new credit card instance.
6
7      The initial balance is zero.
8
9      customer  the name of the customer (e.g., 'John Bowman')
10     bank      the name of the bank (e.g., 'California Savings')
11     acnt      the acount identifier (e.g., '5391 0375 9387 5309')
12     limit     credit limit (measured in dollars)
13     """
14     self._customer = customer
15     self._bank = bank
16     self._account = acnt
17     self._limit = limit
18     self._balance = 0
19
20   def get_customer(self):
21     """Return name of the customer."""
22     return self._customer
23
24   def get_bank(self):
25     """Return the bank's name."""
26     return self._bank
27
28   def get_account(self):
29     """Return the card identifying number (typically stored as a string)."""
30     return self._account
31
32   def get_limit(self):
33     """Return current credit limit."""
34     return self._limit
35
36   def get_balance(self):
37     """Return current balance."""
38     return self._balance
```

Code Fragment 2.1: The beginning of the CreditCard class definition (continued in Code Fragment 2.2).

```
39    def charge(self, price):
40      """Charge given price to the card, assuming sufficient credit limit.
41
42      Return True if charge was processed; False if charge was denied.
43      """
44      if price + self._balance > self._limit:      # if charge would exceed limit,
45        return False                               # cannot accept charge
46      else:
47        self._balance += price
48        return True
49
50    def make_payment(self, amount):
51      """Process customer payment that reduces balance."""
52      self._balance -= amount
```

Code Fragment 2.2: The conclusion of the CreditCard class definition (continued from Code Fragment 2.1). These methods are indented within the class definition.

We draw attention to the difference between the method signature as declared within the class versus that used by a caller. For example, from a user's perspective we have seen that the get_balance method takes zero parameters, yet within the class definition, self is an explicit parameter. Likewise, the charge method is declared within the class having two parameters (self and price), even though this method is called with one parameter, for example, as my_card.charge(200). The interpretter automatically binds the instance upon which the method is invoked to the self parameter.

The Constructor

A user can create an instance of the CreditCard class using a syntax as:

```
cc = CreditCard('John Doe, '1st Bank', '5391 0375 9387 5309', 1000)
```

Internally, this results in a call to the specially named __init__ method that serves as the ***constructor*** of the class. Its primary responsibility is to establish the state of a newly created credit card object with appropriate instance variables. In the case of the CreditCard class, each object maintains five instance variables, which we name: _customer, _bank, _account, _limit, and _balance. The initial values for the first four of those five are provided as explicit parameters that are sent by the user when instantiating the credit card, and assigned within the body of the constructor. For example, the command, self._customer = customer, assigns the instance variable self._customer to the parameter customer; note that because customer is ***unqualified*** on the right-hand side, it refers to the parameter in the local namespace.

Encapsulation

By the conventions described in Section 2.2.3, a single leading underscore in the name of a data member, such as _balance, implies that it is intended as ***nonpublic***. Users of a class should not directly access such members.

As a general rule, we will treat all data members as nonpublic. This allows us to better enforce a consistent state for all instances. We can provide accessors, such as get_balance, to provide a user of our class read-only access to a trait. If we wish to allow the user to change the state, we can provide appropriate update methods. In the context of data structures, encapsulating the internal representation allows us greater flexibility to redesign the way a class works, perhaps to improve the efficiency of the structure.

Additional Methods

The most interesting behaviors in our class are charge and make_payment. The charge function typically adds the given price to the credit card balance, to reflect a purchase of said price by the customer. However, before accepting the charge, our implementation verifies that the new purchase would not cause the balance to exceed the credit limit. The make_payment charge reflects the customer sending payment to the bank for the given amount, thereby reducing the balance on the card. We note that in the command, self._balance $-=$ amount, the expression self._balance is qualified with the self identifier because it represents an instance variable of the card, while the unqualified amount represents the local parameter.

Error Checking

Our implementation of the CreditCard class is not particularly robust. First, we note that we did not explicitly check the types of the parameters to charge and make_payment, nor any of the parameters to the constructor. If a user were to make a call such as visa.charge('candy'), our code would presumably crash when attempting to add that parameter to the current balance. If this class were to be widely used in a library, we might use more rigorous techniques to raise a TypeError when facing such misuse (see Section 1.7).

Beyond the obvious type errors, our implementation may be susceptible to logical errors. For example, if a user were allowed to charge a negative price, such as visa.charge(-300), that would serve to *lower* the customer's balance. This provides a loophole for lowering a balance without making a payment. Of course, this might be considered valid usage if modeling the credit received when a customer returns merchandise to a store. We will explore some such issues with the CreditCard class in the end-of-chapter exercises.

Testing the Class

In Code Fragment 2.3, we demonstrate some basic usage of the CreditCard class, inserting three cards into a list named wallet. We use loops to make some charges and payments, and use various accessors to print results to the console.

These tests are enclosed within a conditional, **if** `__name__ == '__main__'`:, so that they can be embedded in the source code with the class definition. Using the terminology of Section 2.2.4, these tests provide ***method coverage***, as each of the methods is called at least once, but it does not provide ***statement coverage***, as there is never a case in which a charge is rejected due to the credit limit. This is not a particular advanced from of testing as the output of the given tests must be manually audited in order to determine whether the class behaved as expected. Python has tools for more formal testing (see discussion of the unittest module in Section 2.2.4), so that resulting values can be automatically compared to the predicted outcomes, with output generated only when an error is detected.

```
53 if __name__ == '__main__':
54   wallet = [ ]
55   wallet.append(CreditCard('John Bowman', 'California Savings',
56                            '5391 0375 9387 5309', 2500) )
57   wallet.append(CreditCard('John Bowman', 'California Federal',
58                            '3485 0399 3395 1954', 3500) )
59   wallet.append(CreditCard('John Bowman', 'California Finance',
60                            '5391 0375 9387 5309', 5000) )
61
62   for val in range(1, 17):
63     wallet[0].charge(val)
64     wallet[1].charge(2*val)
65     wallet[2].charge(3*val)
66
67   for c in range(3):
68     print('Customer =', wallet[c].get_customer( ))
69     print('Bank =', wallet[c].get_bank( ))
70     print('Account =', wallet[c].get_account( ))
71     print('Limit =', wallet[c].get_limit( ))
72     print('Balance =', wallet[c].get_balance( ))
73     while wallet[c].get_balance( ) > 100:
74       wallet[c].make_payment(100)
75       print('New balance =', wallet[c].get_balance( ))
76     print( )
```

Code Fragment 2.3: Testing the CreditCard class.

2.3.2 Operator Overloading and Python's Special Methods

Python's built-in classes provide natural semantics for many operators. For example, the syntax a + b invokes addition for numeric types, yet concatenation for sequence types. When defining a new class, we must consider whether a syntax like a + b should be defined when a or b is an instance of that class.

By default, the + operator is undefined for a new class. However, the author of a class may provide a definition using a technique known as ***operator overloading***. This is done by implementing a specially named method. In particular, the + operator is overloaded by implementing a method named __add__, which takes the right-hand operand as a parameter and which returns the result of the expression. That is, the syntax, a + b, is converted to a method call on object a of the form, a.__add__(b). Similar specially named methods exist for other operators. Table 2.1 provides a comprehensive list of such methods.

When a binary operator is applied to two instances of different types, as in 3 * 'love me', Python gives deference to the class of the *left* operand. In this example, it would effectively check if the int class provides a sufficient definition for how to multiply an instance by a string, via the __mul__ method. However, if that class does not implement such a behavior, Python checks the class definition for the right-hand operand, in the form of a special method named __rmul__ (i.e., "right multiply"). This provides a way for a new user-defined class to support mixed operations that involve an instance of an existing class (given that the existing class would presumably not have defined a behavior involving this new class). The distinction between __mul__ and __rmul__ also allows a class to define different semantics in cases, such as matrix multiplication, in which an operation is noncommutative (that is, A * x may differ from x * A).

Non-Operator Overloads

In addition to traditional operator overloading, Python relies on specially named methods to control the behavior of various other functionality, when applied to user-defined classes. For example, the syntax, str(foo), is formally a call to the constructor for the string class. Of course, if the parameter is an instance of a user-defined class, the original authors of the string class could not have known how that instance should be portrayed. So the string constructor calls a specially named method, foo.__str__(), that must return an appropriate string representation.

Similar special methods are used to determine how to construct an int, float, or bool based on a parameter from a user-defined class. The conversion to a Boolean value is particularly important, because the syntax, **if** foo:, can be used even when foo is not formally a Boolean value (see Section 1.4.1). For a user-defined class, that condition is evaluated by the special method foo.__bool__().

Common Syntax	Special Method Form	
a + b	a.__add__(b);	alternatively b.__radd__(a)
a − b	a.__sub__(b);	alternatively b.__rsub__(a)
a * b	a.__mul__(b);	alternatively b.__rmul__(a)
a / b	a.__truediv__(b);	alternatively b.__rtruediv__(a)
a // b	a.__floordiv__(b);	alternatively b.__rfloordiv__(a)
a % b	a.__mod__(b);	alternatively b.__rmod__(a)
a ** b	a.__pow__(b);	alternatively b.__rpow__(a)
a << b	a.__lshift__(b);	alternatively b.__rlshift__(a)
a >> b	a.__rshift__(b);	alternatively b.__rrshift__(a)
a & b	a.__and__(b);	alternatively b.__rand__(a)
a ^ b	a.__xor__(b);	alternatively b.__rxor__(a)
a | b	a.__or__(b);	alternatively b.__ror__(a)
a += b a −= b a *= b ...	a.__iadd__(b) a.__isub__(b) a.__imul__(b) ...	
+a	a.__pos__()	
−a	a.__neg__()	
~a	a.__invert__()	
abs(a)	a.__abs__()	
a < b	a.__lt__(b)	
a <= b	a.__le__(b)	
a > b	a.__gt__(b)	
a >= b	a.__ge__(b)	
a == b	a.__eq__(b)	
a != b	a.__ne__(b)	
v in a	a.__contains__(v)	
a[k]	a.__getitem__(k)	
a[k] = v	a.__setitem__(k,v)	
del a[k]	a.__delitem__(k)	
a(arg1, arg2, ...)	a.__call__(arg1, arg2, ...)	
len(a)	a.__len__()	
hash(a)	a.__hash__()	
iter(a)	a.__iter__()	
next(a)	a.__next__()	
bool(a)	a.__bool__()	
float(a)	a.__float__()	
int(a)	a.__int__()	
repr(a)	a.__repr__()	
reversed(a)	a.__reversed__()	
str(a)	a.__str__()	

Table 2.1: Overloaded operations, implemented with Python's special methods.

Several other top-level functions rely on calling specially named methods. For example, the standard way to determine the size of a container type is by calling the top-level len function. Note well that the calling syntax, len(foo), is not the traditional method-calling syntax with the dot operator. However, in the case of a user-defined class, the top-level len function relies on a call to a specially named __len__ method of that class. That is, the call len(foo) is evaluated through a method call, foo.__len__(). When developing data structures, we will routinely define the __len__ method to return a measure of the size of the structure.

Implied Methods

As a general rule, if a particular special method is not implemented in a user-defined class, the standard syntax that relies upon that method will raise an exception. For example, evaluating the expression, a + b, for instances of a user-defined class without __add__ or __radd__ will raise an error.

However, there are some operators that have default definitions provided by Python, in the absence of special methods, and there are some operators whose definitions are derived from others. For example, the __bool__ method, which supports the syntax **if** foo:, has default semantics so that every object other than None is evaluated as True. However, for container types, the __len__ method is typically defined to return the size of the container. If such a method exists, then the evaluation of bool(foo) is interpreted by default to be True for instances with nonzero length, and False for instances with zero length, allowing a syntax such as **if** waitlist: to be used to test whether there are one or more entries in the waitlist.

In Section 2.3.4, we will discuss Python's mechanism for providing iterators for collections via the special method, __iter__. With that said, if a container class provides implementations for both __len__ and __getitem__, a default iteration is provided automatically (using means we describe in Section 2.3.4). Furthermore, once an iterator is defined, default functionality of __contains__ is provided.

In Section 1.3 we drew attention to the distinction between expression a **is** b and expression a == b, with the former evaluating whether identifiers a and b are aliases for the same object, and the latter testing a notion of whether the two identifiers reference *equivalent* values. The notion of "equivalence" depends upon the context of the class, and semantics is defined with the __eq__ method. However, if no implementation is given for __eq__, the syntax a == b is legal with semantics of a **is** b, that is, an instance is equivalent to itself and no others.

We should caution that some natural implications are *not* automatically provided by Python. For example, the __eq__ method supports syntax a == b, but providing that method does not affect the evaluation of syntax a != b. (The __ne__ method should be provided, typically returning **not** (a == b) as a result.) Similarly, providing a __lt__ method supports syntax a < b, and indirectly b > a, but providing both __lt__ and __eq__ does *not* imply semantics for a <= b.

2.3.3 Example: Multidimensional Vector Class

To demonstrate the use of operator overloading via special methods, we provide an implementation of a Vector class, representing the coordinates of a vector in a multidimensional space. For example, in a three-dimensional space, we might wish to represent a vector with coordinates $\langle 5, -2, 3\rangle$. Although it might be tempting to directly use a Python list to represent those coordinates, a list does not provide an appropriate abstraction for a geometric vector. In particular, if using lists, the expression [5, −2, 3] + [1, 4, 2] results in the list [5, −2, 3, 1, 4, 2]. When working with vectors, if $u = \langle 5, -2, 3\rangle$ and $v = \langle 1, 4, 2\rangle$, one would expect the expression, u + v, to return a three-dimensional vector with coordinates $\langle 6, 2, 5\rangle$.

We therefore define a Vector class, in Code Fragment 2.4, that provides a better abstraction for the notion of a geometric vector. Internally, our vector relies upon an instance of a list, named _coords, as its storage mechanism. By keeping the internal list encapsulated, we can enforce the desired public interface for instances of our class. A demonstration of supported behaviors includes the following:

```
v = Vector(5)                  # construct five-dimensional <0, 0, 0, 0, 0>
v[1] = 23                      # <0, 23, 0, 0, 0> (based on use of __setitem__)
v[-1] = 45                     # <0, 23, 0, 0, 45> (also via __setitem__)
print(v[4])                    # print 45 (via __getitem__)
u = v + v                      # <0, 46, 0, 0, 90> (via __add__)
print(u)                       # print <0, 46, 0, 0, 90>
total = 0
for entry in v:                # implicit iteration via __len__ and __getitem__
  total += entry
```

We implement many of the behaviors by trivially invoking a similar behavior on the underlying list of coordinates. However, our implementation of __add__ is customized. Assuming the two operands are vectors with the same length, this method creates a new vector and sets the coordinates of the new vector to be equal to the respective sum of the operands' elements.

It is interesting to note that the class definition, as given in Code Fragment 2.4, automatically supports the syntax u = v + [5, 3, 10, −2, 1], resulting in a new vector that is the element-by-element "sum" of the first vector and the list instance. This is a result of Python's ***polymorphism***. Literally, "polymorphism" means "many forms." Although it is tempting to think of the other parameter of our __add__ method as another Vector instance, we never declared it as such. Within the body, the only behaviors we rely on for parameter other is that it supports len(other) and access to other[j]. Therefore, our code executes when the right-hand operand is a list of numbers (with matching length).

```
1  class Vector:
2    """Represent a vector in a multidimensional space."""
3
4    def __init__(self, d):
5      """Create d-dimensional vector of zeros."""
6      self._coords = [0] * d
7
8    def __len__(self):
9      """Return the dimension of the vector."""
10     return len(self._coords)
11
12   def __getitem__(self, j):
13     """Return jth coordinate of vector."""
14     return self._coords[j]
15
16   def __setitem__(self, j, val):
17     """Set jth coordinate of vector to given value."""
18     self._coords[j] = val
19
20   def __add__(self, other):
21     """Return sum of two vectors."""
22     if len(self) != len(other):              # relies on __len__ method
23       raise ValueError('dimensions must agree')
24     result = Vector(len(self))               # start with vector of zeros
25     for j in range(len(self)):
26       result[j] = self[j] + other[j]
27     return result
28
29   def __eq__(self, other):
30     """Return True if vector has same coordinates as other."""
31     return self._coords == other._coords
32
33   def __ne__(self, other):
34     """Return True if vector differs from other."""
35     return not self == other                 # rely on existing __eq__ definition
36
37   def __str__(self):
38     """Produce string representation of vector."""
39     return '<' + str(self._coords)[1:-1] + '>'   # adapt list representation
```

Code Fragment 2.4: Definition of a simple Vector class.

2.3.4 Iterators

Iteration is an important concept in the design of data structures. We introduced Python's mechanism for iteration in Section 1.8. In short, an ***iterator*** for a collection provides one key behavior: It supports a special method named `__next__` that returns the next element of the collection, if any, or raises a `StopIteration` exception to indicate that there are no further elements.

Fortunately, it is rare to have to directly implement an iterator class. Our preferred approach is the use of the ***generator*** syntax (also described in Section 1.8), which automatically produces an iterator of yielded values.

Python also helps by providing an automatic iterator implementation for any class that defines both `__len__` and `__getitem__`. To provide an instructive example of a low-level iterator, Code Fragment 2.5 demonstrates just such an iterator class that works on any collection that supports both `__len__` and `__getitem__`. This class can be instantiated as `SequenceIterator(data)`. It operates by keeping an internal reference to the data sequence, as well as a current index into the sequence. Each time `__next__` is called, the index is incremented, until reaching the end of the sequence.

```
1  class SequenceIterator:
2    """An iterator for any of Python's sequence types."""
3
4    def __init__(self, sequence):
5      """Create an iterator for the given sequence."""
6      self._seq = sequence              # keep a reference to the underlying data
7      self._k = -1                      # will increment to 0 on first call to next
8
9    def __next__(self):
10     """Return the next element, or else raise StopIteration error."""
11     self._k += 1                      # advance to next index
12     if self._k < len(self._seq):
13       return(self._seq[self._k])      # return the data element
14     else:
15       raise StopIteration( )          # there are no more elements
16
17   def __iter__(self):
18     """By convention, an iterator must return itself as an iterator."""
19     return self
```

Code Fragment 2.5: An iterator class for any sequence type.

2.3.5 Example: Range Class

As the final example for this section, we develop our own implementation of a class that mimics Python's built-in range class. Before introducing our class, we discuss the history of the built-in version. Prior to Python 3 being released, range was implemented as a function, and it returned a list instance with elements in the specified range. For example, range(2, 10, 2) returned the list [2, 4, 6, 8]. However, a typical use of the function was to support a for-loop syntax, such as **for** k **in** range(10000000). Unfortunately, this caused the instantiation and initialization of a list with the range of numbers. That was an unnecessarily expensive step, in terms of both time and memory usage.

The mechanism used to support ranges in Python 3 is entirely different (to be fair, the "new" behavior existed in Python 2 under the name xrange). It uses a strategy known as ***lazy evaluation***. Rather than creating a new list instance, range is a class that can effectively represent the desired range of elements without ever storing them explicitly in memory. To better explore the built-in range class, we recommend that you create an instance as r = range(8, 140, 5). The result is a relatively lightweight object, an instance of the range class, that has only a few behaviors. The syntax len(r) will report the number of elements that are in the given range (27, in our example). A range also supports the __getitem__ method, so that syntax r[15] reports the sixteenth element in the range (as r[0] is the first element). Because the class supports both __len__ and __getitem__, it inherits automatic support for iteration (see Section 2.3.4), which is why it is possible to execute a for loop over a range.

At this point, we are ready to demonstrate our own version of such a class. Code Fragment 2.6 provides a class we name Range (so as to clearly differentiate it from built-in range). The biggest challenge in the implementation is properly computing the number of elements that belong in the range, given the parameters sent by the caller when constructing a range. By computing that value in the constructor, and storing it as self._length, it becomes trivial to return it from the __len__ method. To properly implement a call to __getitem__(k), we simply take the starting value of the range plus k times the step size (i.e., for k=0, we return the start value). There are a few subtleties worth examining in the code:

- To properly support optional parameters, we rely on the technique described on page 27, when discussing a functional version of range.
- We compute the number of elements in the range as
  max(0, (stop − start + step − 1) // step)
  It is worth testing this formula for both positive and negative step sizes.
- The __getitem__ method properly supports negative indices by converting an index −k to len(self)−k before computing the result.

```
1  class Range:
2    """A class that mimic's the built-in range class."""
3
4    def __init__(self, start, stop=None, step=1):
5      """Initialize a Range instance.
6
7      Semantics is similar to built-in range class.
8      """
9      if step == 0:
10       raise ValueError('step cannot be 0')
11
12     if stop is None:                    # special case of range(n)
13       start, stop = 0, start            # should be treated as if range(0,n)
14
15     # calculate the effective length once
16     self._length = max(0, (stop - start + step - 1) // step)
17
18     # need knowledge of start and step (but not stop) to support __getitem__
19     self._start = start
20     self._step = step
21
22   def __len__(self):
23     """Return number of entries in the range."""
24     return self._length
25
26   def __getitem__(self, k):
27     """Return entry at index k (using standard interpretation if negative)."""
28     if k < 0:
29       k += len(self)                    # attempt to convert negative index
30
31     if not 0 <= k < self._length:
32       raise IndexError('index out of range')
33
34     return self._start + k * self._step
```

Code Fragment 2.6: Our own implementation of a Range class.

2.4 Inheritance

A natural way to organize various structural components of a software package is in a ***hierarchical*** fashion, with similar abstract definitions grouped together in a level-by-level manner that goes from specific to more general as one traverses up the hierarchy. An example of such a hierarchy is shown in Figure 2.4. Using mathematical notations, the set of houses is a ***subset*** of the set of buildings, but a ***superset*** of the set of ranches. The correspondence between levels is often referred to as an ***"is a" relationship***, as a house is a building, and a ranch is a house.

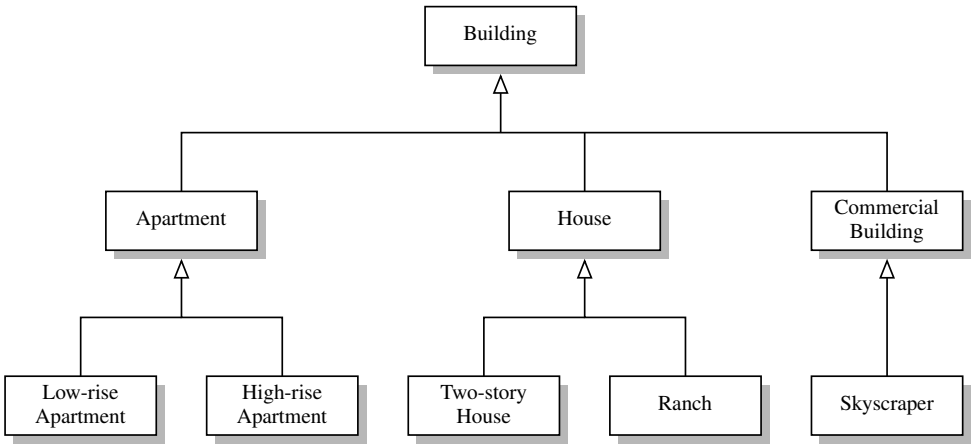


Figure 2.4: An example of an "is a" hierarchy involving architectural buildings.

A hierarchical design is useful in software development, as common functionality can be grouped at the most general level, thereby promoting reuse of code, while differentiated behaviors can be viewed as extensions of the general case, In object-oriented programming, the mechanism for a modular and hierarchical organization is a technique known as ***inheritance***. This allows a new class to be defined based upon an existing class as the starting point. In object-oriented terminology, the existing class is typically described as the ***base class***, ***parent class***, or ***superclass***, while the newly defined class is known as the ***subclass*** or ***child class***.

There are two ways in which a subclass can differentiate itself from its superclass. A subclass may ***specialize*** an existing behavior by providing a new implementation that ***overrides*** an existing method. A subclass may also ***extend*** its superclass by providing brand new methods.

Python's Exception Hierarchy

Another example of a rich inheritance hierarchy is the organization of various exception types in Python. We introduced many of those classes in Section 1.7, but did not discuss their relationship with each other. Figure 2.5 illustrates a (small) portion of that hierarchy. The BaseException class is the root of the entire hierarchy, while the more specific Exception class includes most of the error types that we have discussed. Programmers are welcome to define their own special exception classes to denote errors that may occur in the context of their application. Those user-defined exception types should be declared as subclasses of Exception.

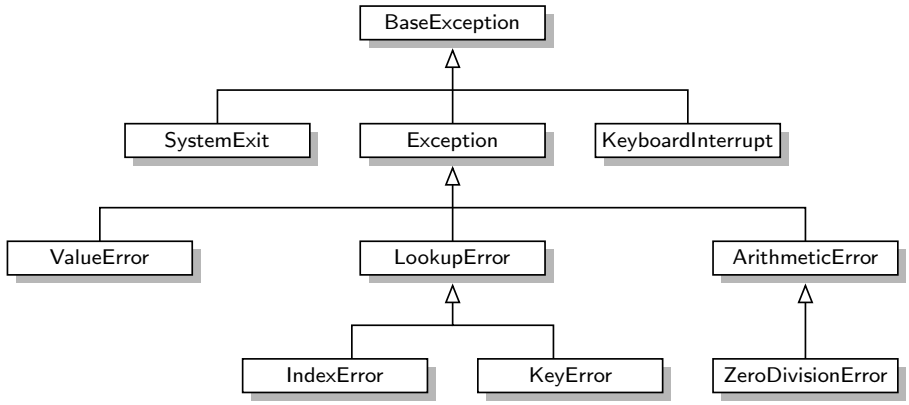


Figure 2.5: A portion of Python's hierarchy of exception types.

2.4.1 Extending the CreditCard Class

To demonstrate the mechanisms for inheritance in Python, we revisit the CreditCard class of Section 2.3, implementing a subclass that, for lack of a better name, we name PredatoryCreditCard. The new class will differ from the original in two ways: (1) if an attempted charge is rejected because it would have exceeded the credit limit, a $5 fee will be charged, and (2) there will be a mechanism for assessing a monthly interest charge on the outstanding balance, based upon an Annual Percentage Rate (APR) specified as a constructor parameter.

In accomplishing this goal, we demonstrate the techniques of specialization and extension. To charge a fee for an invalid charge attempt, we *override* the existing charge method, thereby specializing it to provide the new functionality (although the new version takes advantage of a call to the overridden version). To provide support for charging interest, we extend the class with a new method named process_month.

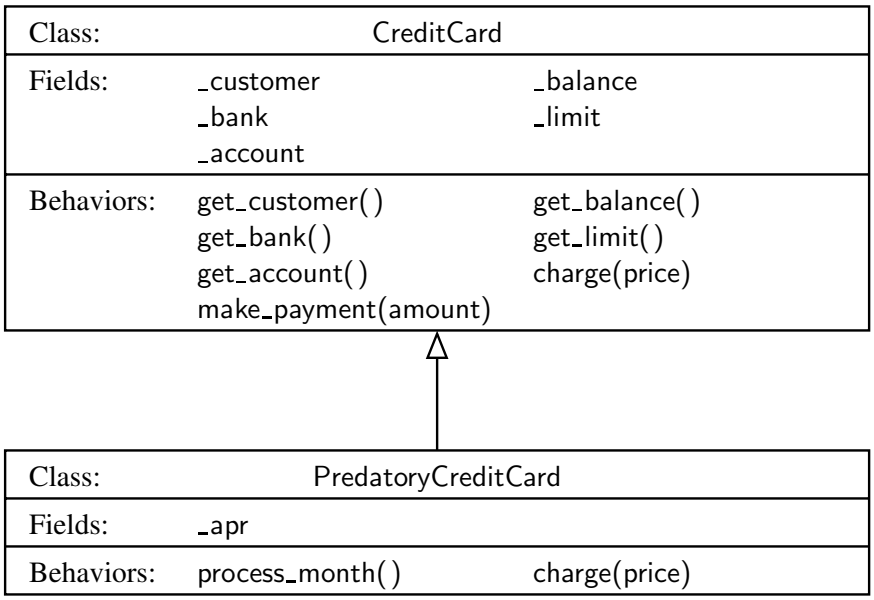


Figure 2.6: Diagram of an inheritance relationship.

Figure 2.6 provides an overview of our use of inheritance in designing the new PredatoryCreditCard class, and Code Fragment 2.7 gives a complete Python implementation of that class.

To indicate that the new class inherits from the existing CreditCard class, our definition begins with the syntax, **class** PredatoryCreditCard(CreditCard). The body of the new class provides three member functions: __init__, charge, and process_month. The __init__ constructor serves a very similar role to the original CreditCard constructor, except that for our new class, there is an extra parameter to specify the annual percentage rate. The body of our new constructor relies upon making a call to the inherited constructor to perform most of the initialization (in fact, everything other than the recording of the percentage rate). The mechanism for calling the inherited constructor relies on the syntax, **super**( ). Specifically, at line 15 the command

super( ).__init__(customer, bank, acnt, limit)

calls the __init__ method that was inherited from the CreditCard superclass. Note well that this method only accepts four parameters. We record the APR value in a new field named _apr.

In similar fashion, our PredatoryCreditCard class provides a new implementation of the charge method that overrides the inherited method. Yet, our implementation of the new method relies on a call to the inherited method, with syntax **super**( ).charge(price) at line 24. The return value of that call designates whether

```
1  class PredatoryCreditCard(CreditCard):
2    """An extension to CreditCard that compounds interest and fees."""
3
4    def __init__(self, customer, bank, acnt, limit, apr):
5      """Create a new predatory credit card instance.
6
7      The initial balance is zero.
8
9      customer  the name of the customer (e.g., 'John Bowman')
10     bank      the name of the bank (e.g., 'California Savings')
11     acnt      the acount identifier (e.g., '5391 0375 9387 5309')
12     limit     credit limit (measured in dollars)
13     apr       annual percentage rate (e.g., 0.0825 for 8.25% APR)
14     """
15     super().__init__(customer, bank, acnt, limit)      # call super constructor
16     self._apr = apr
17
18   def charge(self, price):
19     """Charge given price to the card, assuming sufficient credit limit.
20
21     Return True if charge was processed.
22     Return False and assess $5 fee if charge is denied.
23     """
24     success = super().charge(price)               # call inherited method
25     if not success:
26       self._balance += 5                          # assess penalty
27     return success                                # caller expects return value
28
29   def process_month(self):
30     """Assess monthly interest on outstanding balance."""
31     if self._balance > 0:
32       # if positive balance, convert APR to monthly multiplicative factor
33       monthly_factor = pow(1 + self._apr, 1/12)
34       self._balance *= monthly_factor
```

Code Fragment 2.7: A subclass of CreditCard that assesses interest and fees.

the charge was successful. We examine that return value to decide whether to assess a fee, and in turn we return that value to the caller of method, so that the new version of charge has a similar outward interface as the original.

The process_month method is a new behavior, so there is no inherited version upon which to rely. In our model, this method should be invoked by the bank, once each month, to add new interest charges to the customer's balance. The most challenging aspect in implementing this method is making sure we have working knowledge of how an annual percentage rate translates to a monthly rate. We do not simply divide the annual rate by twelve to get a monthly rate (that would be too predatory, as it would result in a higher APR than advertised). The correct computation is to take the twelfth-root of 1 + self._apr, and use that as a multiplicative factor. For example, if the APR is 0.0825 (representing 8.25%), we compute $\sqrt[12]{1.0825} \approx 1.006628$, and therefore charge 0.6628% interest per month. In this way, each $100 of debt will amass $8.25 of compounded interest in a year.

Protected Members

Our PredatoryCreditCard subclass directly accesses the data member self._balance, which was established by the parent CreditCard class. The underscored name, by convention, suggests that this is a *nonpublic* member, so we might ask if it is okay that we access it in this fashion. While general users of the class should not be doing so, our subclass has a somewhat privileged relationship with the superclass. Several object-oriented languages (e.g., Java, C++) draw a distinction for nonpublic members, allowing declarations of ***protected*** or ***private*** access modes. Members that are declared as protected are accessible to subclasses, but not to the general public, while members that are declared as private are not accessible to either. In this respect, we are using _balance as if it were protected (but not private).

Python does not support formal access control, but names beginning with a single underscore are conventionally akin to protected, while names beginning with a double underscore (other than special methods) are akin to private. In choosing to use protected data, we have created a dependency in that our PredatoryCreditCard class might be compromised if the author of the CreditCard class were to change the internal design. Note that we could have relied upon the public get_balance( ) method to retrieve the current balance within the process_month method. But the current design of the CreditCard class does not afford an effective way for a subclass to change the balance, other than by direct manipulation of the data member. It may be tempting to use charge to add fees or interest to the balance. However, that method does not allow the balance to go above the customer's credit limit, even though a bank would presumably let interest compound beyond the credit limit, if warranted. If we were to redesign the original CreditCard class, we might add a nonpublic method, _set_balance, that could be used by subclasses to affect a change without directly accessing the data member _balance.

2.4.2 Hierarchy of Numeric Progressions

As a second example of the use of inheritance, we develop a hierarchy of classes for iterating numeric progressions. A numeric progression is a sequence of numbers, where each number depends on one or more of the previous numbers. For example, an ***arithmetic progression*** determines the next number by adding a fixed constant to the previous value, and a ***geometric progression*** determines the next number by multiplying the previous value by a fixed constant. In general, a progression requires a first value, and a way of identifying a new value based on one or more previous values.

To maximize reusability of code, we develop a hierarchy of classes stemming from a general base class that we name Progression (see Figure 2.7). Technically, the Progression class produces the progression of whole numbers: 0, 1, 2, ... However, this class is designed to serve as the base class for other progression types, providing as much common functionality as possible, and thereby minimizing the burden on the subclasses.

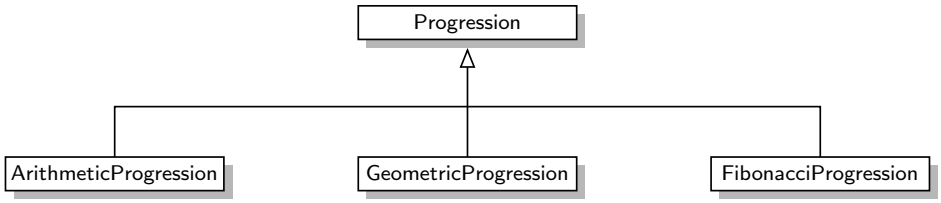


Figure 2.7: Our hierarchy of progression classes.

Our implementation of the basic Progression class is provided in Code Fragment 2.8. The constructor for this class accepts a starting value for the progression (0 by default), and initializes a data member, self._current, to that value.

The Progression class implements the conventions of a Python ***iterator*** (see Section 2.3.4), namely the special __next__ and __iter__ methods. If a user of the class creates a progression as seq = Progression( ), each call to next(seq) will return a subsequent element of the progression sequence. It would also be possible to use a for-loop syntax, **for** value **in** seq:, although we note that our default progression is defined as an infinite sequence.

To better separate the mechanics of the iterator convention from the core logic of advancing the progression, our framework relies on a nonpublic method named _advance to update the value of the self._current field. In the default implementation, _advance adds one to the current value, but our intent is that subclasses will override _advance to provide a different rule for computing the next entry.

For convenience, the Progression class also provides a utility method, named print_progression, that displays the next n values of the progression.

```
class Progression:
  """Iterator producing a generic progression.

  Default iterator produces the whole numbers 0, 1, 2, ...
  """

  def __init__(self, start=0):
    """Initialize current to the first value of the progression."""
    self._current = start

  def _advance(self):
    """Update self._current to a new value.

    This should be overridden by a subclass to customize progression.

    By convention, if current is set to None, this designates the
    end of a finite progression.
    """
    self._current += 1

  def __next__(self):
    """Return the next element, or else raise StopIteration error."""
    if self._current is None:        # our convention to end a progression
      raise StopIteration( )
    else:
      answer = self._current         # record current value to return
      self._advance( )               # advance to prepare for next time
      return answer                  # return the answer

  def __iter__(self):
    """By convention, an iterator must return itself as an iterator."""
    return self

  def print_progression(self, n):
    """Print next n values of the progression."""
    print(' '.join(str(next(self)) for j in range(n)))
```

Code Fragment 2.8: A general numeric progression class.

An Arithmetic Progression Class

Our first example of a specialized progression is an arithmetic progression. While the default progression increases its value by one in each step, an arithmetic progression adds a fixed constant to one term of the progression to produce the next. For example, using an increment of 4 for an arithmetic progression that starts at 0 results in the sequence $0, 4, 8, 12, \ldots$.

Code Fragment 2.9 presents our implementation of an ArithmeticProgression class, which relies on Progression as its base class. The constructor for this new class accepts both an increment value and a starting value as parameters, although default values for each are provided. By our convention, ArithmeticProgression(4) produces the sequence $0, 4, 8, 12, \ldots$, and ArithmeticProgression(4, 1) produces the sequence $1, 5, 9, 13, \ldots$.

The body of the ArithmeticProgression constructor calls the super constructor to initialize the _current data member to the desired start value. Then it directly establishes the new _increment data member for the arithmetic progression. The only remaining detail in our implementation is to override the _advance method so as to add the increment to the current value.

```
1  class ArithmeticProgression(Progression):          # inherit from Progression
2    """Iterator producing an arithmetic progression."""
3
4    def __init__(self, increment=1, start=0):
5      """Create a new arithmetic progression.
6
7      increment  the fixed constant to add to each term (default 1)
8      start      the first term of the progression (default 0)
9      """
10     super().__init__(start)                        # initialize base class
11     self._increment = increment
12
13   def _advance(self):                              # override inherited version
14     """Update current value by adding the fixed increment."""
15     self._current += self._increment
```

Code Fragment 2.9: A class that produces an arithmetic progression.

A Geometric Progression Class

Our second example of a specialized progression is a geometric progression, in which each value is produced by multiplying the preceding value by a fixed constant, known as the ***base*** of the geometric progression. The starting point of a geometric progression is traditionally 1, rather than 0, because multiplying 0 by any factor results in 0. As an example, a geometric progression with base 2 proceeds as $1, 2, 4, 8, 16, \ldots$.

Code Fragment 2.10 presents our implementation of a GeometricProgression class. The constructor uses 2 as a default base and 1 as a default starting value, but either of those can be varied using optional parameters.

```
1  class GeometricProgression(Progression):          # inherit from Progression
2    """Iterator producing a geometric progression."""
3
4    def __init__(self, base=2, start=1):
5      """Create a new geometric progression.
6
7      base        the fixed constant multiplied to each term (default 2)
8      start       the first term of the progression (default 1)
9      """
10     super().__init__(start)
11     self._base = base
12
13   def _advance(self):                             # override inherited version
14     """Update current value by multiplying it by the base value."""
15     self._current *= self._base
```

Code Fragment 2.10: A class that produces a geometric progression.

A Fibonacci Progression Class

As our final example, we demonstrate how to use our progression framework to produce a ***Fibonacci progression***. We originally discussed the Fibonacci series on page 41 in the context of generators. Each value of a Fibonacci series is the sum of the two most recent values. To begin the series, the first two values are conventionally 0 and 1, leading to the Fibonacci series $0, 1, 1, 2, 3, 5, 8, \ldots$. More generally, such a series can be generated from any two starting values. For example, if we start with values 4 and 6, the series proceeds as $4, 6, 10, 16, 26, 42, \ldots$.

```
1  class FibonacciProgression(Progression):
2    """Iterator producing a generalized Fibonacci progression."""
3
4    def __init__(self, first=0, second=1):
5      """Create a new fibonacci progression.
6
7      first        the first term of the progression (default 0)
8      second       the second term of the progression (default 1)
9      """
10     super().__init__(first)                  # start progression at first
11     self._prev = second - first              # fictitious value preceding the first
12
13   def _advance(self):
14     """Update current value by taking sum of previous two."""
15     self._prev, self._current = self._current, self._prev + self._current
```

Code Fragment 2.11: A class that produces a Fibonacci progression.

We use our progression framework to define a new FibonacciProgression class, as shown in Code Fragment 2.11. This class is markedly different from those for the arithmetic and geometric progressions because we cannot determine the next value of a Fibonacci series solely from the current one. We must maintain knowledge of the two most recent values. The base Progression class already provides storage of the most recent value as the _current data member. Our FibonacciProgression class introduces a new member, named _prev, to store the value that proceeded the current one.

With both previous values stored, the implementation of _advance is relatively straightforward. (We use a simultaneous assignment similar to that on page 45.) However, the question arises as to how to initialize the previous value in the constructor. The desired first and second values are provided as parameters to the constructor. The first should be stored as _current so that it becomes the first one that is reported. Looking ahead, once the first value is reported, we will do an assignment to set the new current value (which will be the second value reported), equal to the first value plus the "previous." By initializing the previous value to (second − first), the initial advancement will set the new current value to first + (second − first) = second, as desired.

Testing Our Progressions

To complete our presentation, Code Fragment 2.12 provides a unit test for all of our progression classes, and Code Fragment 2.13 shows the output of that test.

```
if __name__ == '__main__':
  print('Default progression:')
  Progression().print_progression(10)

  print('Arithmetic progression with increment 5:')
  ArithmeticProgression(5).print_progression(10)

  print('Arithmetic progression with increment 5 and start 2:')
  ArithmeticProgression(5, 2).print_progression(10)

  print('Geometric progression with default base:')
  GeometricProgression().print_progression(10)

  print('Geometric progression with base 3:')
  GeometricProgression(3).print_progression(10)

  print('Fibonacci progression with default start values:')
  FibonacciProgression().print_progression(10)

  print('Fibonacci progression with start values 4 and 6:')
  FibonacciProgression(4, 6).print_progression(10)
```

Code Fragment 2.12: Unit tests for our progression classes.

```
Default progression:
0 1 2 3 4 5 6 7 8 9
Arithmetic progression with increment 5:
0 5 10 15 20 25 30 35 40 45
Arithmetic progression with increment 5 and start 2:
2 7 12 17 22 27 32 37 42 47
Geometric progression with default base:
1 2 4 8 16 32 64 128 256 512
Geometric progression with base 3:
1 3 9 27 81 243 729 2187 6561 19683
Fibonacci progression with default start values:
0 1 1 2 3 5 8 13 21 34
Fibonacci progression with start values 4 and 6:
4 6 10 16 26 42 68 110 178 288
```

Code Fragment 2.13: Output of the unit tests from Code Fragment 2.12.

2.4.3 Abstract Base Classes

When defining a group of classes as part of an inheritance hierarchy, one technique for avoiding repetition of code is to design a base class with common functionality that can be inherited by other classes that need it. As an example, the hierarchy from Section 2.4.2 includes a Progression class, which serves as a base class for three distinct subclasses: ArithmeticProgression, GeometricProgression, and FibonacciProgression. Although it is possible to create an instance of the Progression base class, there is little value in doing so because its behavior is simply a special case of an ArithmeticProgression with increment 1. The real purpose of the Progression class was to centralize the implementations of behaviors that other progressions needed, thereby streamlining the code that is relegated to those subclasses.

In classic object-oriented terminology, we say a class is an ***abstract base class*** if its only purpose is to serve as a base class through inheritance. More formally, an abstract base class is one that cannot be directly instantiated, while a ***concrete class*** is one that can be instantiated. By this definition, our Progression class is technically concrete, although we essentially designed it as an abstract base class.

In statically typed languages such as Java and C++, an abstract base class serves as a formal type that may guarantee one or more ***abstract methods***. This provides support for polymorphism, as a variable may have an abstract base class as its declared type, even though it refers to an instance of a concrete subclass. Because there are no declared types in Python, this kind of polymorphism can be accomplished without the need for a unifying abstract base class. For this reason, there is not as strong a tradition of defining abstract base classes in Python, although Python's abc module provides support for defining a formal abstract base class.

Our reason for focusing on abstract base classes in our study of data structures is that Python's collections module provides several abstract base classes that assist when defining custom data structures that share a common interface with some of Python's built-in data structures. These rely on an object-oriented software design pattern known as the ***template method pattern***. The template method pattern is when an abstract base class provides concrete behaviors that rely upon calls to other abstract behaviors. In that way, as soon as a subclass provides definitions for the missing abstract behaviors, the inherited concrete behaviors are well defined.

As a tangible example, the collections.Sequence abstract base class defines behaviors common to Python's list, str, and tuple classes, as sequences that support element access via an integer index. More so, the collections.Sequence class provides concrete implementations of methods, count, index, and __contains__ that can be inherited by any class that provides concrete implementations of both __len__ and __getitem__. For the purpose of illustration, we provide a sample implementation of such a Sequence abstract base class in Code Fragment 2.14.

```
1  from abc import ABCMeta, abstractmethod          # need these definitions
2
3  class Sequence(metaclass=ABCMeta):
4    """Our own version of collections.Sequence abstract base class."""
5
6    @abstractmethod
7    def __len__(self):
8      """Return the length of the sequence."""
9
10   @abstractmethod
11   def __getitem__(self, j):
12     """Return the element at index j of the sequence."""
13
14   def __contains__(self, val):
15     """Return True if val found in the sequence; False otherwise."""
16     for j in range(len(self)):
17       if self[j] == val:                          # found match
18         return True
19     return False
20
21   def index(self, val):
22     """Return leftmost index at which val is found (or raise ValueError)."""
23     for j in range(len(self)):
24       if self[j] == val:                          # leftmost match
25         return j
26     raise ValueError('value not in sequence')    # never found a match
27
28   def count(self, val):
29     """Return the number of elements equal to given value."""
30     k = 0
31     for j in range(len(self)):
32       if self[j] == val:                          # found a match
33         k += 1
34     return k
```

Code Fragment 2.14: An abstract base class akin to collections.Sequence.

This implementation relies on two advanced Python techniques. The first is that we declare the ABCMeta class of the abc module as a ***metaclass*** of our Sequence class. A metaclass is different from a superclass, in that it provides a template for the class definition itself. Specifically, the ABCMeta declaration assures that the constructor for the class raises an error.

The second advanced technique is the use of the @abstractmethod decorator immediately before the __len__ and __getitem__ methods are declared. That declares these two particular methods to be abstract, meaning that we do not provide an implementation within our Sequence base class, but that we expect any concrete subclasses to support those two methods. Python enforces this expectation, by disallowing instantiation for any subclass that does not override the abstract methods with concrete implementations.

The rest of the Sequence class definition provides tangible implementations for other behaviors, under the assumption that the abstract __len__ and __getitem__ methods will exist in a concrete subclass. If you carefully examine the source code, the implementations of methods __contains__, index, and count do not rely on any assumption about the self instances, other than that syntax len(self) and self[j] are supported (by special methods __len__ and __getitem__, respectively). Support for iteration is automatic as well, as described in Section 2.3.4.

In the remainder of this book, we omit the formality of using the abc module. If we need an "abstract" base class, we simply document the expectation that subclasses provide assumed functionality, without technical declaration of the methods as abstract. But we will make use of the wonderful abstract base classes that are defined within the collections module (such as Sequence). To use such a class, we need only rely on standard inheritance techniques.

For example, our Range class, from Code Fragment 2.6 of Section 2.3.5, is an example of a class that supports the __len__ and __getitem__ methods. But that class does not support methods count or index. Had we originally declared it with Sequence as a superclass, then it would also inherit the count and index methods. The syntax for such a declaration would begin as:

```
class Range(collections.Sequence):
```

Finally, we emphasize that if a subclass provides its own implementation of an inherited behaviors from a base class, the new definition overrides the inherited one. This technique can be used when we have the ability to provide a more efficient implementation for a behavior than is achieved by the generic approach. As an example, the general implementation of __contains__ for a sequence is based on a loop used to search for the desired value. For our Range class, there is an opportunity for a more efficient determination of containment. For example, it is evident that the expression, 100000 **in** Range(0, 2000000, 100), should evaluate to True, even without examining the individual elements of the range, because the range starts with zero, has an increment of 100, and goes until 2 million; it must include 100000, as that is a multiple of 100 that is between the start and stop values. Exercise C-2.27 explores the goal of providing an implementation of Range.__contains__ that avoids the use of a (time-consuming) loop.

2.5 Namespaces and Object-Orientation

A ***namespace*** is an abstraction that manages all of the identifiers that are defined in a particular scope, mapping each name to its associated value. In Python, functions, classes, and modules are all first-class objects, and so the "value" associated with an identifier in a namespace may in fact be a function, class, or module.

In Section 1.10 we explored Python's use of namespaces to manage identifiers that are defined with global scope, versus those defined within the local scope of a function call. In this section, we discuss the important role of namespaces in Python's management of object-orientation.

2.5.1 Instance and Class Namespaces

We begin by exploring what is known as the ***instance namespace***, which manages attributes specific to an individual object. For example, each instance of our CreditCard class maintains a distinct balance, a distinct account number, a distinct credit limit, and so on (even though some instances may coincidentally have equivalent balances, or equivalent credit limits). Each credit card will have a dedicated instance namespace to manage such values.

There is a separate ***class namespace*** for each class that has been defined. This namespace is used to manage members that are to be *shared* by all instances of a class, or used without reference to any particular instance. For example, the make_payment method of the CreditCard class from Section 2.3 is not stored independently by each instance of that class. That member function is stored within the namespace of the CreditCard class. Based on our definition from Code Fragments 2.1 and 2.2, the CreditCard class namespace includes the functions: __init__, get_customer, get_bank, get_account, get_balance, get_limit, charge, and make_payment. Our PredatoryCreditCard class has its own namespace, containing the three methods we defined for that subclass: __init__, charge, and process_month.

Figure 2.8 provides a portrayal of three such namespaces: a class namespace containing methods of the CreditCard class, another class namespace with methods of the PredatoryCreditCard class, and finally a single instance namespace for a sample instance of the PredatoryCreditCard class. We note that there are two different definitions of a function named charge, one in the CreditCard class, and then the overriding method in the PredatoryCreditCard class. In similar fashion, there are two distinct __init__ implementations. However, process_month is a name that is only defined within the scope of the PredatoryCreditCard class. The instance namespace includes all data members for the instance (including the _apr member that is established by the PredatoryCreditCard constructor).

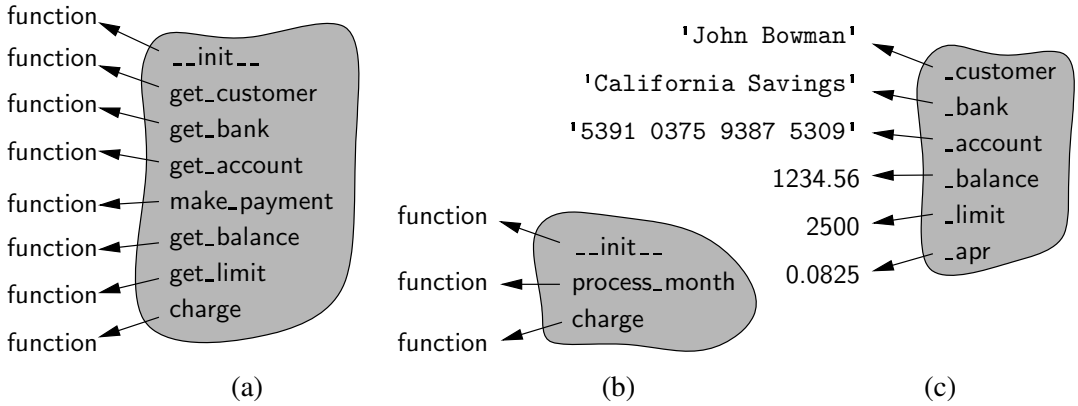


Figure 2.8: Conceptual view of three namespaces: (a) the class namespace for CreditCard; (b) the class namespace for PredatoryCreditCard; (c) the instance namespace for a PredatoryCreditCard object.

How Entries Are Established in a Namespace

It is important to understand why a member such as _balance resides in a credit card's instance namespace, while a member such as make_payment resides in the class namespace. The balance is established within the __init__ method when a new credit card instance is constructed. The original assignment uses the syntax, self._balance = 0, where self is an identifier for the newly constructed instance. The use of self as a qualifier for self._balance in such an assignment causes the _balance identifier to be added directly to the instance namespace.

When inheritance is used, there is still a single *instance namespace* per object. For example, when an instance of the PredatoryCreditCard class is constructed, the _apr attribute as well as attributes such as _balance and _limit all reside in that instance's namespace, because all are assigned using a qualified syntax, such as self._apr.

A *class namespace* includes all declarations that are made directly within the body of the class definition. For example, our CreditCard class definition included the following structure:

```
class CreditCard:
  def make_payment(self, amount):
    ...
```

Because the make_payment function is declared within the scope of the CreditCard class, that function becomes associated with the name make_payment within the CreditCard class namespace. Although member functions are the most typical types of entries that are declared in a class namespace, we next discuss how other types of data values, or even other classes can be declared within a class namespace.

Class Data Members

A class-level data member is often used when there is some value, such as a constant, that is to be shared by all instances of a class. In such a case, it would be unnecessarily wasteful to have each instance store that value in its instance namespace. As an example, we revisit the PredatoryCreditCard introduced in Section 2.4.1. That class assesses a $5 fee if an attempted charge is denied because of the credit limit. Our choice of $5 for the fee was somewhat arbitrary, and our coding style would be better if we used a named variable rather than embedding the literal value in our code. Often, the amount of such a fee is determined by the bank's policy and does not vary for each customer. In that case, we could define and use a class data member as follows:

```
class PredatoryCreditCard(CreditCard):
  OVERLIMIT_FEE = 5                          # this is a class-level member

  def charge(self, price):
    success = super( ).charge(price)
    if not success:
      self._balance += PredatoryCreditCard.OVERLIMIT_FEE
    return success
```

The data member, OVERLIMIT_FEE, is entered into the PredatoryCreditCard class namespace because that assignment takes place within the immediate scope of the class definition, and without any qualifying identifier.

Nested Classes

It is also possible to nest one class definition within the scope of another class. This is a useful construct, which we will exploit several times in this book in the implementation of data structures. This can be done by using a syntax such as

```
class A:          # the outer class
  class B:        # the nested class
    ...
```

In this case, class B is the nested class. The identifier B is entered into the namespace of class A associated with the newly defined class. We note that this technique is unrelated to the concept of inheritance, as class B does not inherit from class A.

Nesting one class in the scope of another makes clear that the nested class exists for support of the outer class. Furthermore, it can help reduce potential name conflicts, because it allows for a similarly named class to exist in another context. For example, we will later introduce a data structure known as a ***linked list*** and will define a nested node class to store the individual components of the list. We will also introduce a data structure known as a ***tree*** that depends upon its own nested

node class. These two structures rely on different node definitions, and by nesting those within the respective container classes, we avoid ambiguity.

Another advantage of one class being nested as a member of another is that it allows for a more advanced form of inheritance in which a subclass of the outer class overrides the definition of its nested class. We will make use of that technique in Section 11.2.1 when specializing the nodes of a tree structure.

Dictionaries and the __slots__ Declaration

By default, Python represents each namespace with an instance of the built-in dict class (see Section 1.2.3) that maps identifying names in that scope to the associated objects. While a dictionary structure supports relatively efficient name lookups, it requires additional memory usage beyond the raw data that it stores (we will explore the data structure used to implement dictionaries in Chapter 10).

Python provides a more direct mechanism for representing instance namespaces that avoids the use of an auxiliary dictionary. To use the streamlined representation for all instances of a class, that class definition must provide a class-level member named __slots__ that is assigned to a fixed sequence of strings that serve as names for instance variables. For example, with our CreditCard class, we would declare the following:

```
class CreditCard:
  __slots__ = '_customer', '_bank', '_account', '_balance', '_limit'
```

In this example, the right-hand side of the assignment is technically a tuple (see discussion of automatic packing of tuples in Section 1.9.3).

When inheritance is used, if the base class declares __slots__, a subclass must also declare __slots__ to avoid creation of instance dictionaries. The declaration in the subclass should only include names of supplemental methods that are newly introduced. For example, our PredatoryCreditCard declaration would include the following declaration:

```
class PredatoryCreditCard(CreditCard):
  __slots__ = '_apr'                    # in addition to the inherited members
```

We could choose to use the __slots__ declaration to streamline every class in this book. However, we do not do so because such rigor would be atypical for Python programs. With that said, there are a few classes in this book for which we expect to have a large number of instances, each representing a lightweight construct. For example, when discussing nested classes, we suggest linked lists and trees as data structures that are often comprised of a large number of individual nodes. To promote greater efficiency in memory usage, we will use an explicit __slots__ declaration in any nested classes for which we expect many instances.

2.5.2 Name Resolution and Dynamic Dispatch

In the previous section, we discussed various namespaces, and the mechanism for establishing entries in those namespaces. In this section, we examine the process that is used when *retrieving* a name in Python's object-oriented framework. When the dot operator syntax is used to access an existing member, such as obj.foo, the Python interpreter begins a name resolution process, described as follows:

1. The instance namespace is searched; if the desired name is found, its associated value is used.
2. Otherwise the class namespace, for the class to which the instance belongs, is searched; if the name is found, its associated value is used.
3. If the name was not found in the immediate class namespace, the search continues upward through the inheritance hierarchy, checking the class namespace for each ancestor (commonly by checking the superclass class, then its superclass class, and so on). The first time the name is found, its associate value is used.
4. If the name has still not been found, an AttributeError is raised.

As a tangible example, let us assume that mycard identifies an instance of the PredatoryCreditCard class. Consider the following possible usage patterns.

- mycard._balance (or equivalently, self._balance from within a method body): the _balance method is found within the *instance namespace* for mycard.
- mycard.process_month( ): the search begins in the instance namespace, but the name process_month is not found in that namespace. As a result, the PredatoryCreditCard class namespace is searched; in this case, the name is found and that method is called.
- mycard.make_payment(200): the search for the name, make_payment, fails in the instance namespace and in the PredatoryCreditCard namespace. The name is resolved in the namespace for superclass CreditCard and thus the inherited method is called.
- mycard.charge(50): the search for name charge fails in the instance namespace. The next namespace checked is for the PredatoryCreditCard class, because that is the true type of the instance. There is a definition for a charge function in that class, and so that is the one that is called.

In the last case shown, notice that the existence of a charge function in the PredatoryCreditCard class has the effect of ***overriding*** the version of that function that exists in the CreditCard namespace. In traditional object-oriented terminology, Python uses what is known as ***dynamic dispatch*** (or ***dynamic binding***) to determine, at run-time, which implementation of a function to call based upon the type of the object upon which it is invoked. This is in contrast to some languages that use ***static dispatching***, making a compile-time decision as to which version of a function to call, based upon the declared type of a variable.

2.6 Shallow and Deep Copying

In Chapter 1, we emphasized that an assignment statement foo = bar makes the name foo an ***alias*** for the object identified as bar. In this section, we consider the task of making a ***copy*** of an object, rather than an alias. This is necessary in applications when we want to subsequently modify either the original or the copy in an independent manner.

Consider a scenario in which we manage various lists of colors, with each color represented by an instance of a presumed color class. We let identifier warmtones denote an existing list of such colors (e.g., oranges, browns). In this application, we wish to create a new list named palette, which is a copy of the warmtones list. However, we want to subsequently be able to add additional colors to palette, or to modify or remove some of the existing colors, without affecting the contents of warmtones. If we were to execute the command

```
palette = warmtones
```

this creates an alias, as shown in Figure 2.9. No new list is created; instead, the new identifier palette references the original list.

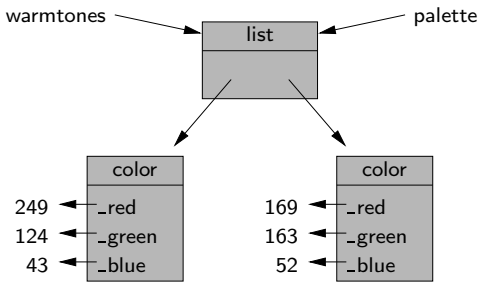


Figure 2.9: Two aliases for the same list of colors.

Unfortunately, this does not meet our desired criteria, because if we subsequently add or remove colors from "palette," we modify the list identified as warmtones.

We can instead create a new instance of the list class by using the syntax:

```
palette = list(warmtones)
```

In this case, we explicitly call the list constructor, sending the first list as a parameter. This causes a new list to be created, as shown in Figure 2.10; however, it is what is known as a ***shallow copy***. The new list is initialized so that its contents are precisely the same as the original sequence. However, Python's lists are ***referential*** (see page 9 of Section 1.2.3), and so the new list represents a sequence of references to the same elements as in the first.

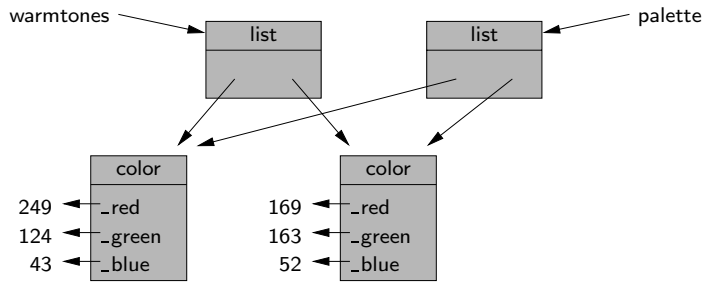


Figure 2.10: A shallow copy of a list of colors.

This is a better situation than our first attempt, as we can legitimately add or remove elements from palette without affecting warmtones. However, if we edit a color instance from the palette list, we effectively change the contents of warmtones. Although palette and warmtones are distinct lists, there remains indirect aliasing, for example, with palette[0] and warmtones[0] as aliases for the same color instance.

We prefer that palette be what is known as a ***deep copy*** of warmtones. In a deep copy, the new copy references its own *copies* of those objects referenced by the original version. (See Figure 2.11.)

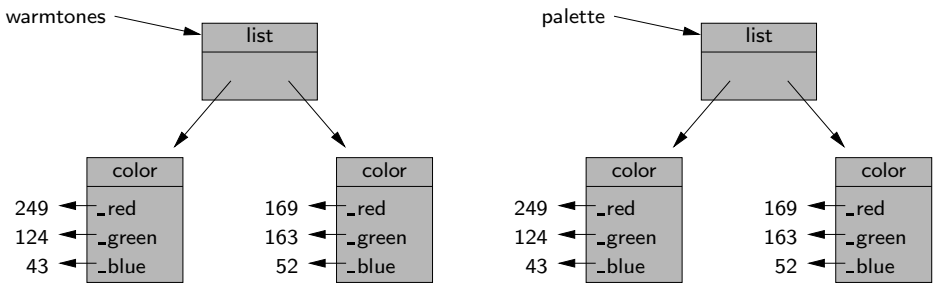


Figure 2.11: A deep copy of a list of colors.

Python's copy Module

To create a deep copy, we could populate our list by explicitly making copies of the original color instances, but this requires that we know how to make copies of colors (rather than aliasing). Python provides a very convenient module, named copy, that can produce both shallow copies and deep copies of arbitrary objects.

This module supports two functions: the copy function creates a shallow copy of its argument, and the deepcopy function creates a deep copy of its argument. After importing the module, we may create a deep copy for our example, as shown in Figure 2.11, using the command:

```
palette = copy.deepcopy(warmtones)
```

2.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-2.1 Give three examples of life-critical software applications.

R-2.2 Give an example of a software application in which adaptability can mean the difference between a prolonged lifetime of sales and bankruptcy.

R-2.3 Describe a component from a text-editor GUI and the methods that it encapsulates.

R-2.4 Write a Python class, Flower, that has three instance variables of type str, int, and float, that respectively represent the name of the flower, its number of petals, and its price. Your class must include a constructor method that initializes each variable to an appropriate value, and your class should include methods for setting the value of each type, and retrieving the value of each type.

R-2.5 Use the techniques of Section 1.7 to revise the charge and make_payment methods of the CreditCard class to ensure that the caller sends a number as a parameter.

R-2.6 If the parameter to the make_payment method of the CreditCard class were a negative number, that would have the effect of *raising* the balance on the account. Revise the implementation so that it raises a ValueError if a negative value is sent.

R-2.7 The CreditCard class of Section 2.3 initializes the balance of a new account to zero. Modify that class so that a new account can be given a nonzero balance using an optional fifth parameter to the constructor. The four-parameter constructor syntax should continue to produce an account with zero balance.

R-2.8 Modify the declaration of the first for loop in the CreditCard tests, from Code Fragment 2.3, so that it will eventually cause exactly one of the three credit cards to go over its credit limit. Which credit card is it?

R-2.9 Implement the __sub__ method for the Vector class of Section 2.3.3, so that the expression u−v returns a new vector instance representing the difference between two vectors.

R-2.10 Implement the __neg__ method for the Vector class of Section 2.3.3, so that the expression −v returns a new vector instance whose coordinates are all the negated values of the respective coordinates of v.

R-2.11 In Section 2.3.3, we note that our Vector class supports a syntax such as $v = u + [5, 3, 10, -2, 1]$, in which the sum of a vector and list returns a new vector. However, the syntax $v = [5, 3, 10, -2, 1] + u$ is illegal. Explain how the Vector class definition can be revised so that this syntax generates a new vector.

R-2.12 Implement the __mul__ method for the Vector class of Section 2.3.3, so that the expression $v * 3$ returns a new vector with coordinates that are 3 times the respective coordinates of v.

R-2.13 Exercise R-2.12 asks for an implementation of __mul__, for the Vector class of Section 2.3.3, to provide support for the syntax $v * 3$. Implement the __rmul__ method, to provide additional support for syntax $3 * v$.

R-2.14 Implement the __mul__ method for the Vector class of Section 2.3.3, so that the expression $u * v$ returns a scalar that represents the dot product of the vectors, that is, $\sum_{i=1}^{d} u_i \cdot v_i$.

R-2.15 The Vector class of Section 2.3.3 provides a constructor that takes an integer d, and produces a d-dimensional vector with all coordinates equal to 0. Another convenient form for creating a new vector would be to send the constructor a parameter that is some iterable type representing a sequence of numbers, and to create a vector with dimension equal to the length of that sequence and coordinates equal to the sequence values. For example, Vector([4, 7, 5]) would produce a three-dimensional vector with coordinates <4, 7, 5>. Modify the constructor so that either of these forms is acceptable; that is, if a single integer is sent, it produces a vector of that dimension with all zeros, but if a sequence of numbers is provided, it produces a vector with coordinates based on that sequence.

R-2.16 Our Range class, from Section 2.3.5, relies on the formula

$$\max(0, (\text{stop} - \text{start} + \text{step} - 1) \mathbin{//} \text{step})$$

to compute the number of elements in the range. It is not immediately evident why this formula provides the correct calculation, even if assuming a positive step size. Justify this formula, in your own words.

R-2.17 Draw a class inheritance diagram for the following set of classes:

- Class Goat extends object and adds an instance variable _tail and methods milk( ) and jump( ).
- Class Pig extends object and adds an instance variable _nose and methods eat(food) and wallow( ).
- Class Horse extends object and adds instance variables _height and _color, and methods run( ) and jump( ).
- Class Racer extends Horse and adds a method race( ).
- Class Equestrian extends Horse, adding an instance variable _weight and methods trot( ) and is_trained( ).

R-2.18 Give a short fragment of Python code that uses the progression classes from Section 2.4.2 to find the 8th value of a Fibonacci progression that starts with 2 and 2 as its first two values.

R-2.19 When using the ArithmeticProgression class of Section 2.4.2 with an increment of 128 and a start of 0, how many calls to next can we make before we reach an integer of 2^{63} or larger?

R-2.20 What are some potential efficiency disadvantages of having very deep inheritance trees, that is, a large set of classes, A, B, C, and so on, such that B extends A, C extends B, D extends C, etc.?

R-2.21 What are some potential efficiency disadvantages of having very shallow inheritance trees, that is, a large set of classes, A, B, C, and so on, such that all of these classes extend a single class, Z?

R-2.22 The collections.Sequence abstract base class does not provide support for comparing two sequences to each other. Modify our Sequence class from Code Fragment 2.14 to include a definition for the __eq__ method, so that expression seq1 == seq2 will return True precisely when the two sequences are element by element equivalent.

R-2.23 In similar spirit to the previous problem, augment the Sequence class with method __lt__, to support lexicographic comparison seq1 < seq2.

Creativity

C-2.24 Suppose you are on the design team for a new e-book reader. What are the primary classes and methods that the Python software for your reader will need? You should include an inheritance diagram for this code, but you do not need to write any actual code. Your software architecture should at least include ways for customers to buy new books, view their list of purchased books, and read their purchased books.

C-2.25 Exercise R-2.12 uses the __mul__ method to support multiplying a Vector by a number, while Exercise R-2.14 uses the __mul__ method to support computing a dot product of two vectors. Give a single implementation of Vector.__mul__ that uses run-time type checking to support both syntaxes u * v and u * k, where u and v designate vector instances and k represents a number.

C-2.26 The SequenceIterator class of Section 2.3.4 provides what is known as a forward iterator. Implement a class named ReversedSequenceIterator that serves as a reverse iterator for any Python sequence type. The first call to next should return the last element of the sequence, the second call to next should return the second-to-last element, and so forth.

C-2.27 In Section 2.3.5, we note that our version of the Range class has implicit support for iteration, due to its explicit support of both __len__ and __getitem__. The class also receives implicit support of the Boolean test, "k **in** r" for Range r. This test is evaluated based on a forward iteration through the range, as evidenced by the relative quickness of the test 2 **in** Range(10000000) versus 9999999 **in** Range(10000000). Provide a more efficient implementation of the __contains__ method to determine whether a particular value lies within a given range. The running time of your method should be independent of the length of the range.

C-2.28 The PredatoryCreditCard class of Section 2.4.1 provides a process_month method that models the completion of a monthly cycle. Modify the class so that once a customer has made ten calls to charge in the current month, each additional call to that function results in an additional $1 surcharge.

C-2.29 Modify the PredatoryCreditCard class from Section 2.4.1 so that a customer is assigned a minimum monthly payment, as a percentage of the balance, and so that a late fee is assessed if the customer does not subsequently pay that minimum amount before the next monthly cycle.

C-2.30 At the close of Section 2.4.1, we suggest a model in which the CreditCard class supports a nonpublic method, _set_balance(b), that could be used by subclasses to affect a change to the balance, without directly accessing the _balance data member. Implement such a model, revising both the CreditCard and PredatoryCreditCard classes accordingly.

C-2.31 Write a Python class that extends the Progression class so that each value in the progression is the absolute value of the difference between the previous two values. You should include a constructor that accepts a pair of numbers as the first two values, using 2 and 200 as the defaults.

C-2.32 Write a Python class that extends the Progression class so that each value in the progression is the square root of the previous value. (Note that you can no longer represent each value with an integer.) Your constructor should accept an optional parameter specifying the start value, using $65,536$ as a default.

Projects

P-2.33 Write a Python program that inputs a polynomial in standard algebraic notation and outputs the first derivative of that polynomial.

P-2.34 Write a Python program that inputs a document and then outputs a bar-chart plot of the frequencies of each alphabet character that appears in that document.

P-2.35 Write a set of Python classes that can simulate an Internet application in which one party, Alice, is periodically creating a set of packets that she wants to send to Bob. An Internet process is continually checking if Alice has any packets to send, and if so, it delivers them to Bob's computer, and Bob is periodically checking if his computer has a packet from Alice, and, if so, he reads and deletes it.

P-2.36 Write a Python program to simulate an ecosystem containing two types of creatures, ***bears*** and ***fish***. The ecosystem consists of a river, which is modeled as a relatively large list. Each element of the list should be a Bear object, a Fish object, or None. In each time step, based on a random process, each animal either attempts to move into an adjacent list location or stay where it is. If two animals of the same type are about to collide in the same cell, then they stay where they are, but they create a new instance of that type of animal, which is placed in a random empty (i.e., previously None) location in the list. If a bear and a fish collide, however, then the fish dies (i.e., it disappears).

P-2.37 Write a simulator, as in the previous project, but add a Boolean gender field and a floating-point strength field to each animal, using an Animal class as a base class. If two animals of the same type try to collide, then they only create a new instance of that type of animal if they are of different genders. Otherwise, if two animals of the same type and gender try to collide, then only the one of larger strength survives.

P-2.38 Write a Python program that simulates a system that supports the functions of an e-book reader. You should include methods for users of your system to "buy" new books, view their list of purchased books, and read their purchased books. Your system should use actual books, which have expired copyrights and are available on the Internet, to populate your set of available books for users of your system to "purchase" and read.

P-2.39 Develop an inheritance hierarchy based upon a Polygon class that has abstract methods area( ) and perimeter( ). Implement classes Triangle, Quadrilateral, Pentagon, Hexagon, and Octagon that extend this base class, with the obvious meanings for the area( ) and perimeter( ) methods. Also implement classes, IsoscelesTriangle, EquilateralTriangle, Rectangle, and Square, that have the appropriate inheritance relationships. Finally, write a simple program that allows users to create polygons of the various types and input their geometric dimensions, and the program then outputs their area and perimeter. For extra effort, allow users to input polygons by specifying their vertex coordinates and be able to test if two such polygons are similar.

Chapter Notes

For a broad overview of developments in computer science and engineering, we refer the reader to *The Computer Science and Engineering Handbook* [96]. For more information about the Therac-25 incident, please see the paper by Leveson and Turner [69].

The reader interested in studying object-oriented programming further, is referred to the books by Booch [17], Budd [20], and Liskov and Guttag [71]. Liskov and Guttag also provide a nice discussion of abstract data types, as does the survey paper by Cardelli and Wegner [23] and the book chapter by Demurjian [33] in the *The Computer Science and Engineering Handbook* [96]. Design patterns are described in the book by Gamma *et al.* [41].

Books with specific focus on object-oriented programming in Python include those by Goldwasser and Letscher [43] at the introductory level, and by Phillips [83] at a more advanced level,